Multi-Modal Grounded Planning and Efficient Replanning For Learning Embodied Agents with A Few Examples

Taro Yamada

Seoul National University
{twoongg.kim, byeonghwikim, jonghyunchoi}@snu.ac.kr

Abstract

Instruction-following embodied agents typically require large-scale language–action paired annotations, which are prohibitively expensive to collect. Large language model (LLM) based planners alleviate this by enabling few-shot planning, yet they frequently produce *ungrounded* plans that ignore the current visual environment, leading to execution failures. We present FLARE (FEW-SHOT LANGUAGE WITH ENVIRONMENTAL ADAPTIVE REPLANNING EMBODIED AGENT), a framework that addresses both limitations. FLARE comprises two key modules: (1) a *Multi-Modal Planner* (MMP) that retrieves in-context examples by combining BERT-based text similarity with CLIP-based image similarity over initial panoramic observations, generating compact subgoal triplets (Action, Object, Receptacle) that reduce prompt length by $\sim$25%; and (2) an *Environment Adaptive Replanning* (EAR) module that, without invoking the LLM, detects execution failures and semantically replaces missing target objects with visually observed alternatives via cosine similarity over language embeddings. Low-level navigation is handled by a deterministic Fast Marching Method (FMM) planner, improving robustness in data-scarce regimes. Evaluated on the ALFRED benchmark with only 0.5% of training data ($\sim$100 examples), FLARE-GPT-4 achieves a Test Unseen Success Rate of 40.88 and Goal Condition score of 51.72, surpassing the prior few-shot state-of-the-art LLM-Planner (Song et al. 2023) by $+24.46$ SR points. Our results demonstrate that multi-modal grounding combined with efficient, LLM-free replanning enables reliable long-horizon task execution with very limited supervision.

1 Introduction

Embodied AI agents that can follow natural language instructions to complete complex, multi-step household tasks have become an important research frontier (Shridhar et al. 2020; Szot et al. 2021; Batra et al. 2020). These tasks require agents to perceive their environment, plan sequences of actions, and interact with objects across many steps. Despite significant progress, a key bottleneck remains: current state-of-the-art methods (Min et al. 2022; Blukis et al. 2021; Kim et al. 2023) rely on thousands of paired language-action demonstrations for training, making them expensive to develop and difficult to adapt to new settings.

Large language models (LLMs) offer a promising path forward for few-shot embodied planning (Brown et al. 2020; Wei et al. 2022). By treating task demonstrations as in-context examples, LLM-based planners such as LLM-Planner (Song et al. 2023) can achieve competitive performance using as little as 0.5% of training data. However, a critical weakness of these approaches is that plans are generated using text alone, without grounding in the *current* visual environment. As a result, LLMs may generate plans that refer to objects not present in the scene or fail to account for the specific spatial layout, leading to downstream execution failures.

We identify two complementary sources of grounding that prior work underutilizes: (1) *visual context at planning time* — the initial panoramic observation of the scene provides rich information about object presence and layout that can guide example retrieval; and (2) *visual feedback at execution time* — when a planned action fails because a target object is absent, the set of currently detected objects provides sufficient signal to repair the plan locally, without re-querying the LLM.

In this paper, we propose FLARE (FEW-SHOT LANGUAGE WITH ENVIRONMENTAL ADAPTIVE REPLANNING EMBODIED AGENT), a framework that exploits both sources of grounding. FLARE consists of two main components. The *Multi-Modal Planner* (MMP) retrieves $k = 9$ in-context examples by computing a weighted combination of BERT-based (Kenton and Toutanova 2019) instruction similarity and CLIP-based (Radford et al. 2021) image similarity over initial panoramic observations, producing plans that are better aligned with the actual scene. Plans are expressed as compact subgoal triplets (Action, Object, Receptacle), reducing prompt length by approximately 25% compared to natural language descriptions. The *Environment Adaptive Replanning* (EAR) module monitors execution and, when a target object is not found, replaces it with the most semantically similar detected object using cosine similarity over language embeddings — entirely without an LLM call, making replanning fast and economical.

Our contributions are as follows:

- A *multi-modal in-context retrieval* strategy that combines textual instruction similarity and CLIP-based visual simi-

Copyright © 2025, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

larity over initial panoramic observations, yielding better-grounded few-shot plans.

- An *LLM-free environment adaptive replanning* module (EAR) that partially corrects subgoals by semantically replacing missing target objects with detected alternatives, avoiding expensive LLM re-invocations.
- A *compact subgoal triplet representation* (Action, Object, Receptacle) that reduces prompt length by $\sim$25% and improves plan interpretability.
- *State-of-the-art few-shot performance* on ALFRED, with FLARE-GPT-4 achieving Test Unseen SR of 40.88 (+24.46 over LLM-Planner) and GC of 51.72 using only $\sim$100 training examples.

2 Related Work

Embodied instruction following. The ALFRED benchmark (Shridhar et al. 2020) has established a standard for evaluating language-conditioned embodied agents in interactive 3D environments. Prior methods such as HLSM (Blukis et al. 2021) and FILM (Min et al. 2022) train end-to-end models on the full dataset and achieve strong performance but require thousands of demonstrations. CAPEAM (Kim et al. 2023) improves data efficiency with context-aware planning and execution, but still depends on substantial training data. In contrast, FLARE targets the few-shot regime, learning from as few as 100 demonstrations.

LLMs for robot task planning. The success of large language models at few-shot reasoning (Brown et al. 2020; Wei et al. 2022) has inspired a wave of work on LLM-driven robot planning. SayCan (Ahn et al. 2022) grounds LLM plans with affordance value functions. ProgPrompt (Singh et al. 2023) prompts LLMs to produce executable code-like plans. Inner Monologue (Huang et al. 2023) integrates environmental feedback into a closed-loop prompting framework. Socratic Models (Zeng et al. 2023) combines multiple foundation models for planning. Code as Policies (Liang et al. 2023) generates robot programs via LLMs. LLM-Planner (Song et al. 2023) — the most closely related prior work — uses text-based in-context retrieval for few-shot embodied planning on ALFRED. FLARE extends this line by adding visual grounding at both the retrieval and replanning stages.

Vision-language models. CLIP (Radford et al. 2021) learns joint image-text representations via contrastive pre-training and has become a standard feature extractor for cross-modal tasks. PaLM-E (Driess et al. 2023) and LLaVA (Liu et al. 2023) integrate visual perception directly into LLMs for multi-modal reasoning. FLARE leverages CLIP for image-based retrieval of in-context examples, grounding the LLM planner in scene-level visual observations without requiring end-to-end multi-modal training.

Replanning and error recovery. Handling execution failures in long-horizon tasks is a persistent challenge for embodied agents. Several works use external feedback to replan with the LLM (Huang et al. 2023; Song et al. 2023), incurring additional API costs. TidyBot (Wu et al. 2023) leverages LLMs to personalize robot plans, but replanning remains LLM-dependent. FLARE's EAR module performs targeted, LLM-free subgoal repair by semantic object substitution, making replanning both fast and cost-efficient.

3 Approach

3.1 Overview

FLARE consists of three main stages as illustrated in Figure 1: (1) multi-modal in-context retrieval and high-level plan generation via MMP; (2) environment adaptive replanning via EAR; and (3) low-level action execution using a deterministic FMM navigation planner. Given a natural language instruction and an initial egocentric panoramic observation of the scene, FLARE generates a high-level plan as a sequence of subgoal triplets, executes them step by step, and adaptively repairs the plan when execution failures are detected.

3.2 Multi-Modal Planner (MMP)

Prior LLM-based planners retrieve in-context examples using only text similarity between the input instruction and training demonstrations (Song et al. 2023). This ignores the visual context of the scene, which can differ dramatically even for semantically similar instructions (e.g., the same "make coffee" task may have very different visual starting states). MMP addresses this by incorporating initial panoramic visual observations into the retrieval process.

Multi-modal retrieval. For each query consisting of an instruction t and an initial panoramic observation v, MMP computes a combined similarity score with each training example (t_i, v_i):

$$s_i = \alpha \cdot \hat{s}_i^{\text{txt}} + \beta \cdot \hat{s}_i^{\text{img}}, \tag{1}$$

where $\hat{s}_i^{\text{txt}}$ and $\hat{s}_i^{\text{img}}$ are the ℓ_2-normalized text and image similarity vectors, respectively. Text similarity is computed using BERT (Kenton and Toutanova 2019) sentence embeddings (`bert-base-nli-mean-tokens`), and image similarity is computed using CLIP (Radford et al. 2021) (`clip-vit-base-patch32`) applied to the initial panoramic observation. We set $\alpha = \beta = 1$ (equal weighting). The top-k most similar training examples (with $k = 9$) are selected as in-context demonstrations for the LLM.

Retrieval hyperparameters. The number of retrieved examples k and the weighting coefficients α, β are important design choices for MMP. We set k=9 based on held-out validation: smaller values (k<6) limit the diversity of demonstrated strategies, while larger values (k>12) introduce dissimilar demonstrations that confuse the LLM planner. Equal weighting (α=β=1) proved robustly effective across ALFRED task types; increasing the visual weight (β>α) benefits spatially distinctive tasks such as `Heat & Place`, where initial scene appearance is highly predictive of the required plan, but can hurt tasks where instruction semantics vary substantially within visually similar scenes. We leave adaptive or learned weighting strategies as future work.

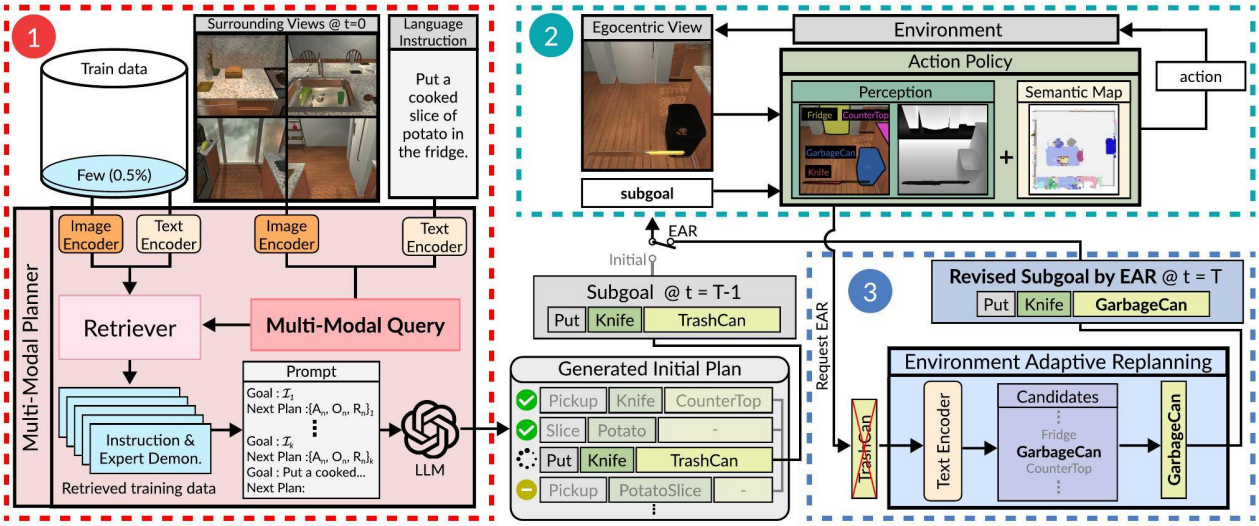

Figure 1: **Detailed architecture of FLARE.** Given a natural language instruction and an initial panoramic observation, the Multi-Modal Planner (MMP) retrieves k in-context examples using combined text and image similarity, then prompts the LLM to produce subgoal triplets. During execution, the Environment Adaptive Replanning (EAR) module replaces missing objects with semantically similar detected alternatives without re-calling the LLM. Low-level navigation is handled by the FMM planner.

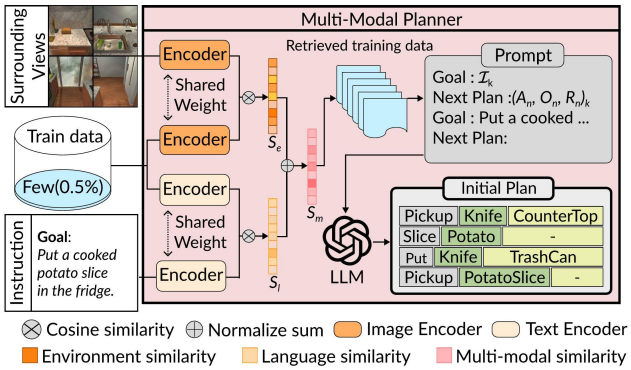

Figure 2: **Multi-Modal Planner.** MMP retrieves in-context examples by combining BERT text similarity with CLIP image similarity over panoramic initial observations, producing better-grounded subgoal triplet plans.

Subgoal triplet representation. Rather than prompting the LLM with natural language descriptions of each step, MMP uses a compact *subgoal triplet* representation: each subgoal is expressed as `(Action, Object, Receptacle)`, where `Action` is one of a fixed set of high-level actions (*e.g.* `GotoLocation`, `PickupObject`, `PutObject`, `HeatObject`, `CoolObject`, `CleanObject`, `SliceObject`, `ToggleObject`), `Object` is the target object, and `Receptacle` is the container or location. This compact format reduces prompt length by approximately 25% compared to natural language descriptions, lowers token usage, and facilitates structured post-processing. Each triplet is deterministically expanded into a sequence of low-level executable actions for environment interaction.

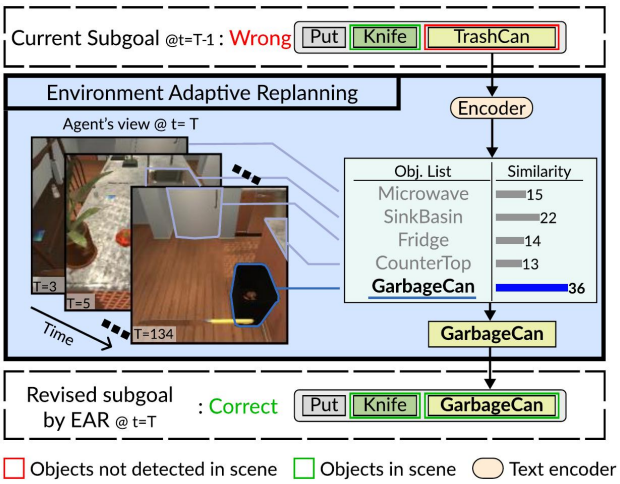

Figure 3: **Environment Adaptive Replanning.** When the target object of a subgoal is not found, EAR replaces it with the most semantically similar detected object using language embedding cosine similarity, without re-invoking the LLM.

3.3 Environment Adaptive Replanning (EAR)

Even with improved retrieval, LLM-generated plans may refer to objects that are not present in the current scene. Prior approaches address this by re-querying the LLM (Song et al. 2023; Huang et al. 2023), which incurs additional API cost and latency. EAR instead performs efficient, *LLM-free* replanning at the subgoal level.

When the agent fails to locate the target object o^* of the current subgoal, EAR retrieves the set $\mathcal{D}$ of objects detected in the current environment via the perception module. It then

selects the replacement object $\hat{o}$ as:

$$\hat{o} = \arg\max_{o \in \mathcal{D}} \cos(\mathbf{e}_{o^*}, \mathbf{e}_o), \quad (2)$$

where $\mathbf{e}_o$ denotes the language embedding of object name o. Only the failed subgoal is updated; the remainder of the plan is preserved, keeping the replanning overhead minimal.

3.4 Low-Level Execution and Navigation

For low-level navigation, FLARE employs the Fast Marching Method (FMM), a deterministic path-planning algorithm (Chaplot et al. 2020). Semantic mapping is used to maintain an occupancy map and object inventory of the environment. The deterministic nature of FMM improves robustness compared to learned navigation policies when training data is scarce, as it requires no data-driven training and generalizes reliably across environments.

3.5 Implementation Details

The perception module uses a Mask R-CNN architecture pretrained on COCO and fine-tuned on AI2-THOR object categories. During execution, object detections are aggregated across frames to maintain a persistent inventory of observed objects throughout the episode. Language embeddings for EAR are computed with a lightweight sentence transformer for fast cosine similarity at runtime. CLIP and BERT encoders operate in inference-only mode with frozen weights; panoramic CLIP encoding adds negligible overhead ($<$50 ms) relative to LLM query latency. The high-level planner queries the LLM only once per task at initialization; all subsequent replanning via EAR requires no additional LLM calls, making FLARE suitable for deployment under rate-limited or cost-sensitive API conditions.

4 Experiments

4.1 Experimental Setup

Benchmark. We evaluate FLARE on ALFRED (Shridhar et al. 2020) (Action Learning From Realistic Environments and Directives), a large-scale benchmark for embodied instruction following in AI2-THOR (Kolve et al. 2017). ALFRED consists of household tasks requiring long sequences of navigation and object interaction actions.

Few-shot setting. Following (Song et al. 2023), we evaluate in the *few-shot* setting where only 0.5% of training data ($\sim$100 examples) is available. We report results on the standard *Test Seen* and *Test Unseen* splits under two instruction modalities: *Goal instructions + Sequential instructions* and *Goal instruction only*.

Metrics. We report Success Rate (SR) and Goal Condition (GC), along with their Path Length Weighted (PLW) variants, which penalize unnecessarily long execution paths.

LLM backends. We evaluate FLARE with four LLM backends: LLaMA2 (Touvron et al. 2023), Vicuna (Zheng et al. 2023), GPT-3.5, and GPT-4 (OpenAI et al. 2024).

Baselines. We compare against: HLSM (Blukis et al. 2021), FILM (Min et al. 2022), CAPEAM (Kim et al. 2023) (trained on full data), and LLM-Planner (Song et al. 2023) (few-shot).

4.2 Comparison with State-of-the-Art

Table 1 reports the main comparison. In the few-shot (0.5%) setting, FLARE-GPT-4 achieves a Test Unseen SR of **40.88** and GC of **51.72** (PLW: 18.14/22.78), compared to LLM-Planner's SR of 16.42 and GC of 23.37 — a gain of **+24.46 SR points**. This large improvement demonstrates the benefit of multi-modal grounding and adaptive replanning in the few-shot regime. Notably, FLARE-GPT-4 also surpasses full-data baselines HLSM (Blukis et al. 2021) (Test Unseen SR 20.27) and FILM (Min et al. 2022) (Test Unseen SR 27.80) on the goal+sequential instruction split, while using only 0.5% of the training data.

Even with weaker LLM backends, FLARE demonstrates consistent gains over LLM-Planner. FLARE-GPT-3.5 achieves Test Unseen SR of 31.79 and GC of 43.94, and FLARE-Vicuna reaches 22.04 SR / 33.57 GC. In the goal-only instruction setting, FLARE-GPT-4 achieves Test Unseen SR of 32.57 and GC of 43.23 (PLW: 12.72/18.40), again leading all few-shot methods.

4.3 Planner Accuracy

To isolate the contribution of the MMP module, Table 2 compares planner accuracy (fraction of generated plans matching the ground-truth plan) between LLM-Planner (static, text-only retrieval) and FLARE (w/o EAR, multi-modal retrieval) for each LLM backend. FLARE consistently and substantially outperforms LLM-Planner across all backends. With GPT-4, FLARE achieves 61.34% Seen and 67.48% Unseen accuracy, compared to 31.54% and 30.12% for LLM-Planner — demonstrating that multi-modal retrieval produces dramatically more accurate high-level plans. Gains are even more pronounced for smaller models; FLARE-LLaMA2 improves from near-zero (0.006% / 0.002%) for LLM-Planner to 18.54% / 22.29%, showing the benefit of visually grounded retrieval for weaker language models.

4.4 Ablation Study

Table 3 presents an ablation study using GPT-3.5 to quantify the individual contributions of MMP and EAR. Removing MMP (row b) reduces Test Unseen SR from 31.79 to 30.35 (-1.44), and removing EAR (row c) reduces it from 31.79 to 30.28 (-1.51). Removing both components (row d) drops performance to 28.58 (-3.21). Both modules contribute independently, and their combination yields the best performance. MMP improves plan quality at generation time, while EAR repairs failures at execution time; together they are complementary and mutually reinforcing.

4.5 Discussion

A key practical advantage of FLARE is its modularity: MMP and EAR are decoupled and compatible with any LLM backend. The EAR module avoids LLM re-invocations

Table 1: **Comparison with state-of-the-art methods.** The path-length-weighteed (PLW) metrics are presented in the parentheses for each metric. †We excerpt 'SR' and 'GC' from (Song et al. 2023). For models that did not report the PLW metric, we noted 'N/A' in our comparison.

Setting	Model	Goal instructions + Sequential instructions				Goal instruction only			
		Test Seen		Test Unseen		Test Seen		Test Unseen	
		SR	GC	SR	GC	SR	GC	SR	GC
Few-shot (0.5%)	HLSM (Blukis et al. 2021)†	0.82 (N/A)	6.88 (N/A)	0.61 (N/A)	3.72 (N/A)	N/A	N/A	N/A	N/A
	FILM (Min et al. 2022)†	0.00 (N/A)	4.23 (N/A)	0.20 (N/A)	6.71 (N/A)	N/A	N/A	N/A	N/A
	CAPEAM (Kim et al. 2023)	0.00 (0.00)	3.90 (2.29)	0.20 (0.00)	6.63 (2.36)	N/A	N/A	N/A	N/A
	LLM-Planner (Song et al. 2023)	18.20 (N/A)	26.77 (N/A)	16.42 (N/A)	23.37 (N/A)	15.33 (N/A)	24.57 (N/A)	13.41 (N/A)	22.89 (N/A)
	FLARE-LLaMA2	16.96 (4.60)	24.84 (8.09)	17.79 (5.62)	27.40 (9.46)	12.00 (3.01)	20.05 (7.22)	13.73 (4.27)	21.98 (8.46)
	FLARE-Vicuna	20.61 (6.28)	29.57 (10.17)	22.04 (7.61)	33.57 (12.06)	16.37 (4.57)	23.68 (8.84)	18.05 (5.98)	26.75 (10.75)
	FLARE-GPT-3.5	32.55 (12.17)	42.02 (16.94)	31.79 (12.21)	43.94 (17.44)	23.48 (8.71)	33.40 (14.40)	25.38 (9.37)	36.02 (15.28)
	FLARE-GPT-4 (Ours)	**40.05 (16.68)**	**48.84 (21.31)**	**40.88 (18.14)**	**51.72 (22.78)**	**31.96 (12.93)**	**41.36 (18.55)**	**32.57 (12.72)**	**43.23 (18.40)**
Full	HLSM (Blukis et al. 2021)	29.94 (8.74)	41.21 (14.58)	20.27 (5.55)	30.31 (9.99)	25.11 (6.69)	35.79 (11.53)	16.29 (4.34)	27.24 (8.45)
	FILM (Min et al. 2022)	28.83 (11.27)	39.55 (15.59)	27.80 (11.32)	38.52 (15.13)	25.77 (10.39)	36.15 (14.17)	24.46 (9.67)	34.75 (13.13)
	CAPEAM (Kim et al. 2023)	51.79 (21.60)	60.50 (25.88)	46.11 (19.45)	57.33(24.06)	47.36 (19.03)	54.38 (23.78)	43.69 (17.64)	55.66 (22.76)

Table 2: **Planner accuracy comparison.** 'Seen Acc.' and 'Unseen Acc.' denote planner accuracies in valid seen and unseen folds. A plan is regarded as correct when it is aligned with the ground-truth plan. To solely compare planner accuracy, we omit replanning in LLM-Planner and FLARE, denoted with 'Static' and 'w/o EAR'.

LLM	Method	Seen Acc.	Unseen Acc.
LLaMA2	LLM-Planner (Static) (Song et al. 2023)	0.006	0.002
LLaMA2	**FLARE (w/o EAR)**	**18.54**	**22.29**
Vicuna	LLM-Planner (Static) (Song et al. 2023)	8.17	7.06
Vicuna	**FLARE (w/o EAR)**	**24.51**	**33.62**
GPT-3.5	LLM-Planner (Static) (Song et al. 2023)	29.78	31.67
GPT-3.5	**FLARE (w/o EAR)**	**46.10**	**55.66**
GPT-4	LLM-Planner (Static) (Song et al. 2023)	31.54	30.12
GPT-4	**FLARE (w/o EAR)**	**61.34**	**67.48**

Table 3: **Ablation study.** PLW metrics are presented in the parentheses for each metric. MMP and EAR each denotes 'Multi-Modal Planner' and 'Environment Adaptive Replanning,' respectively. We observe that each component contributes to agent's performance.

#	MMP	EAR	Test Seen		Test Unseen	
			SR	GC	SR	GC
(*a*)	✓	✓	32.55 (12.17)	42.02 (16.94)	31.79 (12.21)	43.94 (17.44)
(*b*)	✗	✓	30.20 (12.13)	41.26 (17.27)	30.35 (11.62)	42.40 (16.66)
(*c*)	✓	✗	30.79 (11.98)	40.20 (16.51)	30.28 (12.01)	42.48 (17.03)
(*d*)	✗	✗	28.05 (11.48)	38.64 (16.23)	28.58 (11.82)	39.92 (16.13)

entirely; in our experiments it activates on approximately 30% of subgoals in unseen environments, preventing a corresponding API call each time and reducing end-to-end latency. The compact subgoal triplet representation further facilitates integration with symbolic executors, making FLARE a practical building block for data-efficient embodied AI beyond ALFRED.

Failure analysis reveals two primary error sources. First, *perceptual errors* occur when the object detector fails to detect an object that is present in the scene, causing EAR to substitute an incorrect alternative. Second, *plan ambiguity* arises when instructions admit multiple valid interpretations, reducing the quality of retrieved in-context examples even under multi-modal retrieval. Extending MMP to incorporate multi-view or video-level visual representations for richer scene grounding, and augmenting EAR with active search heuristics, are promising directions for mitigating these failure modes in future work.

Task-type analysis. ALFRED comprises seven task types (*e.g.* `Pick&Place`, `Heat & Place`, `Cool & Place`, `Clean & Place`, `Stack & Place`, `Pick Two & Place`, and `Look At Obj In Light`), which vary substantially in plan length and interaction complexity. Multi-step tasks such as `Heat & Place` benefit most from MMP's visual grounding, as the initial panoramic observation often reveals the presence of heat-source appliances and target objects, enabling more accurate in-context retrieval. Shorter tasks such as `Pick&Place` are more frequently assisted by EAR, since their dominant failure mode is object absence rather than multi-step sequencing errors. These observations indicate that MMP and EAR address complementary failure modes across the ALFRED task taxonomy, and suggest that future work could tailor retrieval and replanning strategies to specific task categories for further gains.

The interaction between MMP and EAR reveals an important synergy: better-grounded initial plans produced by MMP reduce the frequency of EAR activations, since fewer subgoals reference absent objects. Conversely, EAR acts as a safety net that tolerates residual grounding errors in MMP's output. This complementarity is evident in the ablation results, where removing either module degrades performance and removing both causes a larger-than-additive drop. The robustness of this pipeline also stems from its asymmetric use of the LLM: costly generative inference is confined to plan initialization, while execution-time corrections rely on lightweight embedding lookups. This separation of concerns — expressive planning at initialization and efficient local correction during execution — mirrors classical hierarchical planning architectures and suggests a broadly applicable design principle for LLM-based agents operating under computational or budget constraints.

The relationship between planner accuracy and task success is not strictly monotonic, as errors at different stages of the execution pipeline carry different downstream consequences. A plan with a single incorrect subgoal may still yield a high goal condition score if the remaining subgoals are correctly executed and the failed step corresponds to a non-critical interaction. EAR partially mitigates error propagation by replacing failed subgoals with semantically similar alternatives and recovering task progress without a full replan. However, early-stage failures — such as an incorrect

Figure 4: **Benefits of proposed Multi-Modal Planner (MMP).** Multi-modal retrieval produces plans that better match the actual scene layout compared to text-only retrieval, reducing object mismatch errors.

initial navigation target — tend to compound across subsequent subgoals, degrading success more severely than late-stage errors. This asymmetry suggests a priority-weighted replanning strategy, in which EAR applies more conservative substitution criteria for early subgoals and more permissive criteria for later ones, as a promising direction for future investigation.

4.6 Efficiency Analysis

FLARE is designed to minimize LLM query overhead while maximizing task performance. Because the LLM is invoked only once per episode during initialization, runtime costs scale with episode length only through deterministic operations: CLIP panoramic encoding ($<$50 ms on a standard GPU), FMM path planning, and EAR's cosine similarity lookup ($<$1 ms per invocation). Compared to replanning baselines that re-query the LLM upon each execution failure (Song et al. 2023; Huang et al. 2023), FLARE incurs substantially lower API costs, as EAR substitutions require no network round-trips. The compact subgoal triplet representation further reduces prompt token counts by $\sim$25%, directly lowering per-query cost and latency relative to natural-language plan prompts. These properties make FLARE particularly suited for budget-constrained or rate-limited deployment settings, where frequent LLM re-invocations are impractical, and for long-horizon tasks where the number of potential failure points — and thus replanning calls — grows with plan length.

4.7 Qualitative Results

Figure 4 illustrates how MMP produces better-grounded plans by incorporating visual observations: compared

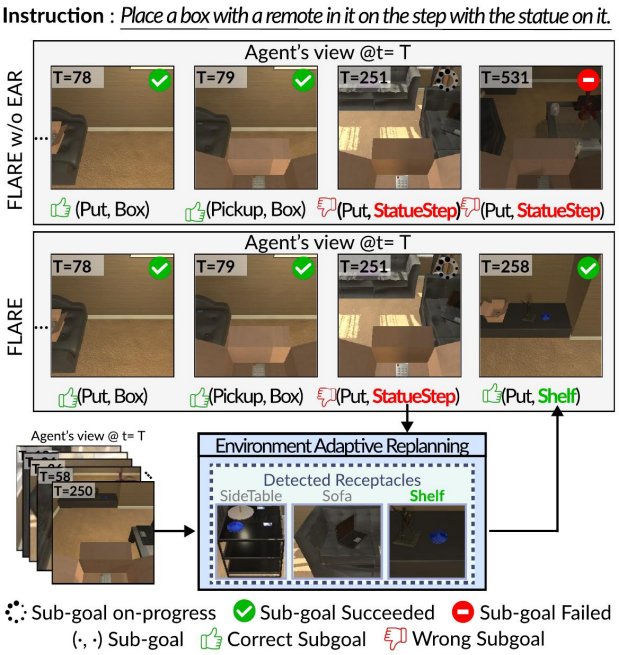

Figure 5: **Benefits of proposed Environment Adaptive Replanning (EAR).** When the planned target object is absent, EAR substitutes a semantically similar detected alternative, allowing the agent to complete the task without LLM re-invocation.

to text-only retrieval, the retrieved demonstrations more closely match the visual context of the query scene. Figure 5 shows a representative EAR case: the plan refers to a `Mug` not present in the scene, and EAR substitutes it with a semantically similar `Cup` that is detected, enabling successful task completion. These examples highlight the complementary nature of the two modules.

5 Conclusion

We presented FLARE, a few-shot embodied agent that combines multi-modal grounded planning with efficient, LLM-free replanning. The Multi-Modal Planner (MMP) retrieves in-context examples using both textual and visual (CLIP-based) similarity over initial panoramic observations, producing plans that are more faithful to the current scene. The Environment Adaptive Replanning (EAR) module handles execution failures by semantically substituting missing objects with detected alternatives, without re-invoking the LLM. Together, these components yield substantial performance gains on ALFRED, with FLARE-GPT-4 surpassing the prior few-shot state-of-the-art by more than 24 SR points using only 100 training examples. Ablations confirm that both MMP and EAR contribute independently and synergistically to overall performance. We believe the principles underlying FLARE — grounding plans in multi-modal observations and performing efficient local replanning — generalize beyond ALFRED and provide a practical foundation for building versatile, data-efficient embodied agents in diverse real-world environments.

References

Ahn, M.; Brohan, A.; Brown, N.; Chebotar, Y.; Cortes, O.; David, B.; Finn, C.; Fu, C.; Gopalakrishnan, K.; Hausman, K.; et al. 2022. Do as i can, not as i say: Grounding language in robotic affordances. In *CoRL*.

Batra, D.; Gokaslan, A.; Kembhavi, A.; Maksymets, O.; Mottaghi, R.; Savva, M.; Toshev, A.; and Wijmans, E. 2020. ObjectNav Revisited: On Evaluation of Embodied Agents Navigating to Objects. In *arXiv:2006.13171*.

Blukis, V.; Paxton, C.; Fox, D.; Garg, A.; and Artzi, Y. 2021. A Persistent Spatial Semantic Representation for High-level Natural Language Instruction Execution. In *CoRL*.

Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; Agarwal, S.; Herbert-Voss, A.; Krueger, G.; Henighan, T.; Child, R.; Ramesh, A.; Ziegler, D.; Wu, J.; Winter, C.; Hesse, C.; Chen, M.; Sigler, E.; Litwin, M.; Gray, S.; Chess, B.; Clark, J.; Berner, C.; McCandlish, S.; Radford, A.; Sutskever, I.; and Amodei, D. 2020. Language Models are Few-Shot Learners. In *NeurIPS*.

Chaplot, D. S.; Gandhi, D.; Gupta, S.; Gupta, A.; and Salakhutdinov, R. 2020. Learning to explore using active neural slam. *arXiv preprint arXiv:2004.05155*.

Driess, D.; Xia, F.; Sajjadi, M. S. M.; Lynch, C.; Chowdhery, A.; Ichter, B.; Wahid, A.; Tompson, J.; Vuong, Q.; Yu, T.; Huang, W.; Chebotar, Y.; Sermanet, P.; Duckworth, D.; Levine, S.; Vanhoucke, V.; Hausman, K.; Toussaint, M.; Greff, K.; Zeng, A.; Mordatch, I.; and Florence, P. 2023. PaLM-E: An Embodied Multimodal Language Model. In *ICML*.

Huang, W.; Xia, F.; Xiao, T.; Chan, H.; Liang, J.; Florence, P.; Zeng, A.; Tompson, J.; Mordatch, I.; Chebotar, Y.; Sermanet, P.; Jackson, T.; Brown, N.; Luu, L.; Levine, S.; Hausman, K.; and Ichter, B. 2023. Inner Monologue: Embodied Reasoning through Planning with Language Models. In *CoRL*.

Kenton, J. D. M.-W. C.; and Toutanova, L. K. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT*.

Kim, B.; Kim, J.; Kim, Y.; Min, C.; and Choi, J. 2023. Context-Aware Planning and Environment-Aware Memory for Instruction Following Embodied Agents. In *ICCV*.

Kolve, E.; Mottaghi, R.; Han, W.; VanderBilt, E.; Weihs, L.; Herrasti, A.; Gordon, D.; Zhu, Y.; Gupta, A.; and Farhadi, A. 2017. AI2-THOR: An Interactive 3D Environment for Visual AI. *arXiv:1712.05474*.

Liang, J.; Huang, W.; Xia, F.; Xu, P.; Hausman, K.; Ichter, B.; Florence, P.; and Zeng, A. 2023. Code as policies: Language model programs for embodied control. In *ICRA*.

Liu, H.; Li, C.; Wu, Q.; and Lee, Y. J. 2023. Visual Instruction Tuning. In *NeurIPS*.

Min, S. Y.; Chaplot, D. S.; Ravikumar, P.; Bisk, Y.; and Salakhutdinov, R. 2022. FILM: Following Instructions in Language with Modular Methods. In *ICLR*.

OpenAI; Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; Avila, R.; Babuschkin, I.; Balaji, S.; Balcom, V.; Baltescu, P.; Bao, H.; Bavarian, M.; Belgum, J.; Bello, I.; Berdine, J.; Bernadett-Shapiro, G.; Berner, C.; Bogdonoff, L.; Boiko, O.; Boyd, M.; Brakman, A.-L.; Brockman, G.; Brooks, T.; Brundage, M.; Button, K.; Cai, T.; Campbell, R.; Cann, A.; Carey, B.; Carlson, C.; Carmichael, R.; Chan, B.; Chang, C.; Chantzis, F.; Chen, D.; Chen, S.; Chen, R.; Chen, J.; Chen, M.; Chess, B.; Cho, C.; Chu, C.; Chung, H. W.; Cummings, D.; Currier, J.; Dai, Y.; Decareaux, C.; Degry, T.; Deutsch, N.; Deville, D.; Dhar, A.; Dohan, D.; Dowling, S.; Dunning, S.; Ecoffet, A.; Eleti, A.; Eloundou, T.; Farhi, D.; Fedus, L.; Felix, N.; Fishman, S. P.; Forte, J.; Fulford, I.; Gao, L.; Georges, E.; Gibson, C.; Goel, V.; Gogineni, T.; Goh, G.; Gontijo-Lopes, R.; Gordon, J.; Grafstein, M.; Gray, S.; Greene, R.; Gross, J.; Gu, S. S.; Guo, Y.; Hallacy, C.; Han, J.; Harris, J.; He, Y.; Heaton, M.; Heidecke, J.; Hesse, C.; Hickey, A.; Hickey, W.; Hoeschele, P.; Houghton, B.; Hsu, K.; Hu, S.; Hu, X.; Huizinga, J.; Jain, S.; Jain, S.; Jang, J.; Jiang, A.; Jiang, R.; Jin, H.; Jin, D.; Jomoto, S.; Jonn, B.; Jun, H.; Kaftan, T.; Łukasz Kaiser; Kamali, A.; Kanitscheider, I.; Keskar, N. S.; Khan, T.; Kilpatrick, L.; Kim, J. W.; Kim, C.; Kim, Y.; Kirchner, J. H.; Kiros, J.; Knight, M.; Kokotajlo, D.; Łukasz Kondraciuk; Kondrich, A.; Konstantinidis, A.; Kosic, K.; Krueger, G.; Kuo, V.; Lampe, M.; Lan, I.; Lee, T.; Leike, J.; Leung, J.; Levy, D.; Li, C. M.; Lim, R.; Lin, M.; Lin, S.; Litwin, M.; Lopez, T.; Lowe, R.; Lue, P.; Makanju, A.; Malfacini, K.; Manning, S.; Markov, T.; Markovski, Y.; Martin, B.; Mayer, K.; Mayne, A.; McGrew, B.; McKinney, S. M.; McLeavey, C.; McMillan, P.; McNeil, J.; Medina, D.; Mehta, A.; Menick, J.; Metz, L.; Mishchenko, A.; Mishkin, P.; Monaco, V.; Morikawa, E.; Mossing, D.; Mu, T.; Murati, M.; Murk, O.; Mély, D.; Nair, A.; Nakano, R.; Nayak, R.; Neelakantan, A.; Ngo, R.; Noh, H.; Ouyang, L.; O'Keefe, C.; Pachocki, J.; Paino, A.; Palermo, J.; Pantuliano, A.; Parascandolo, G.; Parish, J.; Parparita, E.; Passos, A.; Pavlov, M.; Peng, A.; Perelman, A.; de Avila Belbute Peres, F.; Petrov, M.; de Oliveira Pinto, H. P.; Michael; Pokorny; Pokrass, M.; Pong, V. H.; Powell, T.; Power, A.; Power, B.; Proehl, E.; Puri, R.; Radford, A.; Rae, J.; Ramesh, A.; Raymond, C.; Real, F.; Rimbach, K.; Ross, C.; Rotsted, B.; Roussez, H.; Ryder, N.; Saltarelli, M.; Sanders, T.; Santurkar, S.; Sastry, G.; Schmidt, H.; Schnurr, D.; Schulman, J.; Selsam, D.; Sheppard, K.; Sherbakov, T.; Shieh, J.; Shoker, S.; Shyam, P.; Sidor, S.; Sigler, E.; Simens, M.; Sitkin, J.; Slama, K.; Sohl, I.; Sokolowsky, B.; Song, Y.; Staudacher, N.; Such, F. P.; Summers, N.; Sutskever, I.; Tang, J.; Tezak, N.; Thompson, M. B.; Tillet, P.; Tootoonchian, A.; Tseng, E.; Tuggle, P.; Turley, N.; Tworek, J.; Uribe, J. F. C.; Vallone, A.; Vijayvergiya, A.; Voss, C.; Wainwright, C.; Wang, J. J.; Wang, A.; Wang, B.; Ward, J.; Wei, J.; Weinmann, C.; Welihinda, A.; Welinder, P.; Weng, J.; Weng, L.; Wiethoff, M.; Willner, D.; Winter, C.; Wolrich, S.; Wong, H.; Workman, L.; Wu, S.; Wu, J.; Wu, M.; Xiao, K.; Xu, T.; Yoo, S.; Yu, K.; Yuan, Q.; Zaremba, W.; Zellers, R.; Zhang, C.; Zhang, M.; Zhao, S.; Zheng, T.; Zhuang, J.; Zhuk, W.; and Zoph, B. 2024. GPT-4 Technical Report. *arXiv2303.08774*.

Radford, A.; Kim, J. W.; Hallacy, C.; Ramesh, A.; Goh, G.; Agarwal, S.; Sastry, G.; Askell, A.; Mishkin, P.; Clark, J.;

Krueger, G.; and Sutskever, I. 2021. Learning transferable visual models from natural language supervision. In *ICML*.

Shridhar, M.; Thomason, J.; Gordon, D.; Bisk, Y.; Han, W.; Mottaghi, R.; Zettlemoyer, L.; and Fox, D. 2020. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *CVPR*.

Singh, I.; Blukis, V.; Mousavian, A.; Goyal, A.; Xu, D.; Tremblay, J.; Fox, D.; Thomason, J.; and Garg, A. 2023. Progprompt: Generating situated robot task plans using large language models. In *ICRA*.

Song, C. H.; Wu, J.; Washington, C.; Sadler, B. M.; Chao, W.-L.; and Su, Y. 2023. LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with Large Language Models. In *ICCV*.

Szot, A.; Clegg, A.; Undersander, E.; Wijmans, E.; Zhao, Y.; Turner, J.; Maestre, N.; Mukadam, M.; Chaplot, D.; Maksymets, O.; Gokaslan, A.; Vondrus, V.; Dharur, S.; Meier, F.; Galuba, W.; Chang, A.; Kira, Z.; Koltun, V.; Malik, J.; Savva, M.; and Batra, D. 2021. Habitat 2.0: Training Home Assistants to Rearrange their Habitat. In *NeurIPS*.

Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv:2307.09288*.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; Zhou, D.; et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*.

Wu, J.; Antonova, R.; Kan, A.; Lepert, M.; Zeng, A.; Song, S.; Bohg, J.; Rusinkiewicz, S.; and Funkhouser, T. 2023. Tidybot: Personalized robot assistance with large language models. In *IROS*.

Zeng, A.; Attarian, M.; Ichter, B.; Choromanski, K.; Wong, A.; Welker, S.; Tombari, F.; Purohit, A.; Ryoo, M.; Sindhwani, V.; et al. 2023. Socratic models: Composing zero-shot multimodal reasoning with language. In *ICLR*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E.; et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. In *NeurIPS Datasets and Benchmarks Track*.